DEVOPS NETWORKING

THE COMPLETE MASTERY GUIDE

From Zero to Interview-Ready — Never Forget This Again

TOPICS COVERED

Chapter 1: IP Addressing

ANALOGY: Your IP address is like your home address. Just as the postal system needs your exact address to deliver mail, the internet needs your IP to deliver data. Without an address, no one can find you, and you can't receive anything.

1.1 What is an IP Address?

An IP (Internet Protocol) address is a unique numerical label assigned to every device on a network. It serves TWO purposes: identifying the host/device, and providing the location (network) of that device.

Version	Format & Details
IPv4	32-bit address. Format: 4 octets separated by dots. Example: 192.168.1.10. ~4.3 billion addresses total.
IPv6	128-bit address. Format: 8 groups of 4 hex digits. Example: 2001:0db8:85a3::8a2e:0370:7334. ~340 undecillion addresses.

1.2 IP Address Classes (IPv4)

IPv4 addresses are divided into classes. As a DevOps engineer, you must know this cold:

Class	Range | Default Subnet | Use Case
Class A	1.0.0.0 – 126.255.255.255 | /8 (255.0.0.0) | Large networks (ISPs, big corps)
Class B	128.0.0.0 – 191.255.255.255 | /16 (255.255.0.0) | Medium networks
Class C	192.0.0.0 – 223.255.255.255 | /24 (255.255.255.0) | Small networks (most common)
Class D	224.0.0.0 – 239.255.255.255 | N/A | Multicast
Class E	240.0.0.0 – 255.255.255.255 | N/A | Experimental/Reserved
127.x.x.x	Loopback — 127.0.0.1 is always 'this machine itself' (localhost)

1.3 Private vs Public IP Addresses

CRITICAL DISTINCTION: Private IPs are used inside networks (your home, office, cloud VPC). Public IPs are used on the internet. Private IPs are NOT routable on the internet.

⚡ CHEAT SHEET: Private IP Ranges (RFC 1918)	
10.0.0.0/8	10.0.0.0 – 10.255.255.255 → 16 million addresses. Used by AWS VPCs, large orgs.
172.16.0.0/12	172.16.0.0 – 172.31.255.255 → Used by Docker by default.
192.168.0.0/16	192.168.0.0 – 192.168.255.255 → Your home router. Most familiar range.

1.4 CIDR Notation — The Most Important Concept

ANALOGY: CIDR is like saying 'deliver to any house on Oak Street' vs a specific address. The /24 means all 256 houses in that neighborhood. The /32 means exactly ONE house.

CIDR (Classless Inter-Domain Routing) uses a /prefix to define how many bits are the NETWORK portion. The rest are HOST bits.

⚡ CHEAT SHEET: CIDR Quick Reference (MEMORIZE THIS)	
/32	1 IP address. Used for a single host route or AWS security group rule.
/31	2 IPs. Used for point-to-point links.
/30	4 IPs, 2 usable. Smallest practical subnet.
/29	8 IPs, 6 usable.
/28	16 IPs, 14 usable.
/27	32 IPs, 30 usable.
/26	64 IPs, 62 usable.
/25	128 IPs, 126 usable.
/24	256 IPs, 254 usable. Classic Class C subnet. Home network.
/23	512 IPs, 510 usable.
/22	1,024 IPs. Common for medium cloud VPC subnets.
/16	65,536 IPs. Default Docker bridge. AWS VPC typical size.
/8	16,777,216 IPs. Entire Class A network.
/0	All IPs — the entire internet. Used in default routes.

The Formula

Number of IPs = 2^(32 - prefix). Usable hosts = IPs - 2 (subtract network address and broadcast address).

```
# Example: 192.168.1.0/24
Network address : 192.168.1.0   (first — represents the network itself)
First usable    : 192.168.1.1
Last usable     : 192.168.1.254
Broadcast       : 192.168.1.255 (last — sends to ALL hosts in subnet)
Total IPs       : 256  (2^8 = 256)
Usable hosts    : 254  (256 - 2)
```

1.5 Subnetting

Subnetting is dividing a large network into smaller sub-networks. WHY? Security isolation, traffic control, efficient IP use, and organizational structure.

ANALOGY: Subnetting is like dividing a city into neighborhoods. All neighborhoods share the city's address (network), but each neighborhood has its own local streets (hosts). Mail (traffic) inside a neighborhood doesn't need to go to the post office (router).

```
# Real-world DevOps Example: AWS VPC Subnetting
VPC CIDR: 10.0.0.0/16  (65,536 IPs)

Public Subnets (internet-facing):
  10.0.1.0/24  → us-east-1a public  (web servers, load balancers)
  10.0.2.0/24  → us-east-1b public

Private Subnets (no direct internet):
  10.0.11.0/24 → us-east-1a private (app servers, containers)
  10.0.12.0/24 → us-east-1b private

Database Subnets (most isolated):
  10.0.21.0/24 → us-east-1a db (RDS, ElastiCache)
  10.0.22.0/24 → us-east-1b db
```

1.6 NAT — Network Address Translation

NAT allows multiple devices with PRIVATE IPs to share ONE public IP when accessing the internet. This is how your entire office or home network uses one public IP.

HOW IT WORKS	Your device sends packet from private IP (10.0.0.5:3456) → NAT device replaces source IP with public IP (203.0.113.1:54321) → Internet server responds to public IP → NAT translates back and forwards to 10.0.0.5.

NAT Type	Description & DevOps Use
SNAT (Source NAT)	Changes the source IP. Used when private hosts access the internet. Your home router does this.
DNAT (Destination NAT)	Changes the destination IP. Used for port forwarding, ingress to services. Load balancers use this.
Masquerade	Dynamic SNAT — used when public IP changes (dial-up, DHCP). Linux iptables feature.

🎯 INTERVIEW QUESTIONS YOU WILL BE ASKED
Q1: What is the difference between a public and private IP address?
Q2: Explain CIDR notation. What does /24 mean? How many hosts can it hold?
Q3: What is NAT and why is it needed?
Q4: What IP range does Docker use by default? Why might this cause conflicts?
Q5: What is the loopback address? What happens when you ping 127.0.0.1?

Q6: Given 192.168.10.0/26, how many subnets and hosts are available?
Q7: What is the difference between IPv4 and IPv6? Why are we migrating?

Chapter 2: TCP/IP — The Language of the Internet

ANALOGY: TCP/IP is like the entire postal system. IP is the addressing system (zip codes, street addresses). TCP is the reliable delivery guarantee — like certified mail with a signature. UDP is like dropping a postcard in a mailbox — fast, no confirmation.

2.1 The OSI Model vs TCP/IP Model

Two models describe how networks work. OSI has 7 layers (theoretical). TCP/IP has 4 layers (practical). DevOps engineers work mostly at layers 3, 4, and 7.

OSI Layer	TCP/IP Layer | Protocol Examples | DevOps Relevance
7 - Application	Application | HTTP, HTTPS, DNS, SSH, FTP | APIs, web apps, microservices
6 - Presentation	Application | TLS/SSL, encoding | Encryption, data format
5 - Session	Application | Sockets, sessions | Connection management
4 - Transport	Transport | TCP, UDP | Ports, reliability, load balancing
3 - Network	Internet | IP, ICMP, routing | Subnets, routing, VPCs
2 - Data Link	Network Access | Ethernet, MAC, ARP | Switch-level, VLANs
1 - Physical	Network Access | Cables, WiFi, fiber | Hardware layer

MEMORY TRICK OSI 7 layers from top: 'All People Seem To Need Data Processing' (Application, Presentation, Session, Transport, Network, Data Link, Physical)

2.2 TCP — Transmission Control Protocol

TCP is connection-oriented and reliable. Before any data is sent, a connection is established. Every packet is acknowledged. Lost packets are retransmitted.

The TCP 3-Way Handshake (CRITICAL — appears in every interview)

```
Client                          Server
  |                               |
  |-------- SYN (seq=x) -------->|  Step 1: Client says 'Hey, want to talk?'
  |                               |
  |<-- SYN-ACK (seq=y, ack=x+1) -|  Step 2: Server says 'Yes! Ready. Here's my
seq.'
  |                               |
  |-------- ACK (ack=y+1) ------->|  Step 3: Client says 'Got it. Let's go!'
  |                               |
  |======= DATA TRANSFER =========|  Connection established!
```

TCP 4-Way Termination

```
Client                          Server
  |-------- FIN ----------------->|  'I'm done sending.'
  |<-------- ACK -----------------|  'Got it.'
  |<-------- FIN -----------------|  'I'm done sending too.'
  |--------- ACK ---------------->|  'Got it. Bye!'
  |   (Client waits TIME_WAIT)    |
```

Important TCP Flags

⚡ CHEAT SHEET: TCP Flags	
SYN	Synchronize — initiates connection
ACK	Acknowledge — confirms receipt
FIN	Finish — gracefully closes connection
RST	Reset — abruptly closes connection (error or security)
PSH	Push — send data to app immediately without buffering
URG	Urgent — prioritize this data

2.3 UDP — User Datagram Protocol

UDP is connectionless and fast. No handshake. No acknowledgment. Fire and forget. Used when speed matters more than reliability.

Feature	TCP vs UDP
Connection	TCP: Connection-oriented (handshake) | UDP: Connectionless (no handshake)
Reliability	TCP: Guaranteed delivery, retransmission | UDP: Best effort, may drop packets
Order	TCP: Packets arrive in order | UDP: Order not guaranteed
Speed	TCP: Slower (overhead) | UDP: Faster (no overhead)
Use Cases	TCP: HTTP, HTTPS, SSH, FTP, email | UDP: DNS, video streaming, gaming, VoIP
Header Size	TCP: 20 bytes minimum | UDP: 8 bytes

ANALOGY: TCP is like a phone call — you establish a connection, confirm the other person can hear you, then talk, and say goodbye. UDP is like shouting across a room — you just yell and hope they heard you.

2.4 Ports — The Door Numbers

If an IP address is a building, a PORT is a specific door. Ports allow one machine to run many services simultaneously. Range: 0–65535.

⚡ CHEAT SHEET: Essential Ports Every DevOps Engineer Knows

20/21 - FTP	File Transfer Protocol (data/control). Legacy, avoid in production.
22 - SSH	Secure Shell. Your most-used port. Encrypted remote access.
23 - Telnet	Unencrypted remote access. NEVER use — everything is plaintext.
25 - SMTP	Email sending (server-to-server).
53 - DNS	Domain Name System. Both UDP (queries) and TCP (zone transfers).
80 - HTTP	Web traffic, unencrypted.
443 - HTTPS	Web traffic, TLS encrypted. Your default web port.
3000	Default for many dev servers (Node.js, Grafana).
3306 - MySQL	MySQL database. Never expose to public!
5432 - PostgreSQL	PostgreSQL database. Same — keep private.
5672 - AMQP	RabbitMQ message broker.
6379 - Redis	Redis in-memory cache/database.
8080/8443	Alternative HTTP/HTTPS. Common for apps behind a reverse proxy.
9090	Prometheus metrics server.
9200 - Elasticsearch	Elasticsearch HTTP API.
27017 - MongoDB	MongoDB. Keep private.

PORT RANGES	Well-Known: 0–1023 (require root). Registered: 1024–49151 (applications). Ephemeral/Dynamic: 49152–65535 (OS-assigned for outgoing connections).

2.5 ICMP — The Diagnostic Protocol

ICMP (Internet Control Message Protocol) is used for network diagnostics and error reporting. It's NOT for data transfer.

⚡ CHEAT SHEET: ICMP Tools You Use Daily	
ping	Sends ICMP Echo Request, expects Echo Reply. Tests basic connectivity and latency.
traceroute/tracert	Uses ICMP TTL exceeded messages to map route to destination.
TTL (Time To Live)	Each router decrements TTL by 1. Reaches 0? Router drops packet and sends ICMP TTL Exceeded. Prevents infinite loops.

```
# Useful diagnostics
ping -c 4 8.8.8.8              # Test connectivity to Google DNS
traceroute google.com           # See every router hop to destination
ss -tulpn                       # List all listening ports (modern netstat)
netstat -tulpn                  # List listening ports (classic)
curl -v https://example.com     # HTTP request with headers (debug)
telnet 10.0.1.5 80              # Test TCP port connectivity
nc -zv 10.0.1.5 22             # Netcat port check
```

🎯 INTERVIEW QUESTIONS YOU WILL BE ASKED
Q1: Explain the TCP 3-way handshake. What happens if SYN-ACK is never received?
Q2: What is the difference between TCP and UDP? When would you use UDP?
Q3: What port does SSH run on? HTTPS? MySQL?
Q4: What is TTL and what happens when it reaches zero?
Q5: What does a TCP RST mean vs FIN?
Q6: What is an ephemeral port? Why does the OS need them?
Q7: How would you check which process is listening on port 8080?

Chapter 3: DNS — The Internet's Phone Book

ANALOGY: DNS is the internet's phone book. You know 'google.com' but computers need IP addresses. DNS translates names to numbers, just like looking up a name in a directory to find their phone number.

3.1 How DNS Resolution Works (Step by Step)

When you type 'www.example.com' in your browser, here is EXACTLY what happens:

```
Step 1: Browser checks its own DNS cache
        → Found? Done. Not found? Continue.

Step 2: OS checks /etc/hosts file
        → Custom local mappings (127.0.0.1 localhost, etc.)

Step 3: OS asks Recursive Resolver (usually your ISP or 8.8.8.8)
        → Resolver checks its own cache
        → Found? Return it. Not found? Continue.

Step 4: Resolver asks Root Name Server (13 clusters worldwide)
        → 'I don't know example.com, but .com is handled by Verisign'
        → Returns address of .com TLD nameserver

Step 5: Resolver asks .com TLD Server
        → 'I don't know example.com specifically, but ns1.example.com handles it'
        → Returns address of example.com's authoritative nameserver

Step 6: Resolver asks Authoritative Nameserver for example.com
        → 'www.example.com is at 93.184.216.34'
        → FINAL ANSWER

Step 7: Resolver caches the result (for TTL seconds) and returns to browser
Step 8: Browser connects to 93.184.216.34
```

3.2 DNS Record Types

DNS records are entries in a zone file. You MUST know all of these for DevOps work:

Record Type	Purpose & Example
A	Maps domain → IPv4 address. example.com → 93.184.216.34. Most common record.
AAAA	Maps domain → IPv6 address. example.com → 2606:2800:220:1:248:1893:25c8:1946
CNAME	Alias — maps one domain to ANOTHER domain (not IP). www.example.com → example.com. Cannot be used at zone apex!
MX	Mail Exchange — where to deliver email. Priority + mail server hostname.
TXT	Text record — used for SPF, DKIM, domain verification. AWS ACM, Google uses this.

NS	Name Server — authoritative servers for the domain.
SOA	Start of Authority — primary nameserver, zone admin email, TTL settings.
PTR	Reverse DNS — maps IP → domain. Used in email (spam prevention) and security logs.
SRV	Service locator — protocol, port, host. Used by Kubernetes, SIP, XMPP.
CAA	Certification Authority Authorization — which CAs can issue certs for your domain.
ALIAS/ANAME	AWS-specific: like CNAME but works at zone apex. Use for apex domain → ALB.

3.3 TTL — Time To Live

TTL tells DNS resolvers how long to cache a record (in seconds). This is CRITICAL for DevOps operations — especially deployments and incident response.

GOLDEN RULE	BEFORE a deployment or IP change: lower TTL to 60 seconds (wait for old TTL to expire first). AFTER deployment is stable: raise TTL back to 300-3600 for performance. During an incident: low TTL = fast recovery.

⚡ CHEAT SHEET: TTL Values & When to Use Them	
60 seconds	Pre-change / emergency. Fast propagation but high DNS server load.
300 seconds (5 min)	Good default for dynamic environments.
3600 seconds (1 hr)	Standard for semi-static records.
86400 seconds (24 hr)	For very stable records (MX, NS). Don't use for A records you might need to change.

3.4 DNS in DevOps Contexts

Service Discovery

In microservices, containers are constantly created/destroyed. DNS is the backbone of service discovery. Kubernetes uses CoreDNS. Docker uses an embedded DNS server.

```
# Kubernetes DNS format:
<service-name>.<namespace>.svc.cluster.local

# Examples:
mysql.database.svc.cluster.local       # MySQL in 'database' namespace
redis.caching.svc.cluster.local        # Redis in 'caching' namespace
api.default.svc.cluster.local          # API in 'default' namespace

# Debug DNS inside a pod:
kubectl exec -it debug-pod -- nslookup mysql.database.svc.cluster.local
kubectl exec -it debug-pod -- cat /etc/resolv.conf
```

Split-Horizon DNS

Same domain name returns DIFFERENT IP addresses depending on WHO is asking. Internal users get private IPs. External users get public IPs. Critical in hybrid cloud setups.

🎯 INTERVIEW QUESTIONS YOU WILL BE ASKED
Q1: Walk me through DNS resolution from typing a URL to getting an IP.
Q2: What is the difference between A and CNAME records? When can't you use CNAME?
Q3: Why would you lower TTL before a major deployment?
Q4: What is a PTR record used for?
Q5: How does Kubernetes DNS work? What format do service DNS names follow?
Q6: What is the difference between authoritative DNS and recursive resolver?
Q7: What file on Linux is checked before DNS? What does /etc/resolv.conf do?

Chapter 4: HTTP & HTTPS — How the Web Talks

ANALOGY: HTTP is like a conversation between a customer (browser) and a waiter (server). You ask for something (request), they bring it (response). HTTPS is the same conversation but in a soundproof, encrypted booth so no one can eavesdrop.

4.1 HTTP Request/Response Lifecycle

```
# HTTP Request
GET /api/users HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGci...
Accept: application/json
Content-Type: application/json

# HTTP Response
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: max-age=3600
X-Request-ID: a1b2c3d4

{"users": [{"id": 1, "name": "Alice"}]}
```

4.2 HTTP Status Codes — Know These Cold

Code Range	Meaning & Key Codes
1xx — Informational	100 Continue, 101 Switching Protocols (WebSocket upgrade)
2xx — Success	200 OK, 201 Created, 204 No Content (DELETE success), 206 Partial Content
3xx — Redirect	301 Moved Permanently, 302 Found (temp redirect), 304 Not Modified (cache hit)
4xx — Client Error	400 Bad Request, 401 Unauthorized (no/bad creds), 403 Forbidden (wrong perms), 404 Not Found, 405 Method Not Allowed, 408 Timeout, 409 Conflict, 429 Too Many Requests
5xx — Server Error	500 Internal Server Error, 502 Bad Gateway (upstream dead), 503 Service Unavailable (overloaded), 504 Gateway Timeout (upstream too slow)

CRITICAL DevOps	502 = Your upstream service is DOWN (app crashed, wrong port). 503 = Service is up but overwhelmed. 504 = Service is up but SLOW. These distinctions matter during incident response!

4.3 HTTP Methods

⚡ CHEAT SHEET: HTTP Methods (REST)	
GET	Read data. Idempotent. Should have NO side effects. Params in URL.

POST	Create resource. NOT idempotent. Body contains data. Returns 201.
PUT	Replace entire resource. Idempotent. Send full object.
PATCH	Partial update. Send only changed fields.
DELETE	Remove resource. Returns 204 (no content) typically.
HEAD	Like GET but returns ONLY headers, no body. Used for health checks.
OPTIONS	Returns allowed methods. Used in CORS preflight requests.

4.4 HTTPS & TLS — How Encryption Works

HTTPS = HTTP + TLS (Transport Layer Security). TLS replaced SSL (SSL is deprecated and insecure). TLS provides: confidentiality (encryption), integrity (tamper detection), and authentication (identity verification).

TLS Handshake (Simplified)

```
Client                                Server
  |                                     |
  |--- ClientHello (TLS version,  --->|  'Hi, here are ciphers I support'
  |    supported ciphers, random)      |
  |                                     |
  |<-- ServerHello (chosen cipher, ---|  'OK, let's use AES-256. Here's my cert.'
  |    certificate, random)            |
  |                                     |
  | [Client verifies cert against CA] |
  |                                     |
  |--- Key Exchange ----------------->|  'Here's encrypted pre-master secret'
  |                                     |
  | Both sides derive session keys    |
  |                                     |
  |<========= ENCRYPTED DATA =======-->|
```

TLS Certificates

Cert Type	Use Case
DV (Domain Validated)	Cheapest. Only proves domain ownership. Let's Encrypt uses this. Fine for most apps.
OV (Org Validated)	Proves org exists. More trust.
EV (Extended Validation)	Highest trust. Shows company name in browser. Banks use this.
Wildcard (*.example.com)	Covers all subdomains. One cert for api.x.com, www.x.com, etc.
SAN (Multi-domain)	Covers multiple different domains in one cert.
Self-signed	No CA. Used internally or in dev. Browsers show warning — never use in production.

4.5 HTTP/1.1 vs HTTP/2 vs HTTP/3

Version	Key Improvements
HTTP/1.1	Persistent connections (keep-alive). Still sequential — each request waits. Head-of-line blocking.
HTTP/2	Multiplexing (multiple requests in ONE connection simultaneously). Header compression (HPACK). Binary protocol. Server push. Major performance improvement.
HTTP/3	Uses QUIC (UDP-based) instead of TCP. Faster handshake. Better on lossy networks (mobile). Eliminates TCP head-of-line blocking.

4.6 Important HTTP Headers for DevOps

⚡ CHEAT SHEET: Headers You Must Know	
Authorization	Bearer <token> or Basic base64(user:pass). Authentication.
Content-Type	application/json, text/html, multipart/form-data. What format is the body?
Cache-Control	max-age=3600, no-cache, no-store. Controls caching behavior.
CORS Headers	Access-Control-Allow-Origin: * — controls cross-origin requests.
X-Forwarded-For	Original client IP when behind load balancer/proxy. Your app sees proxy IP, not client.
X-Request-ID	Unique ID per request for distributed tracing. Add to every service.
Strict-Transport-Security	Forces HTTPS. Prevents downgrade attacks. Add to all production services.
Set-Cookie	Sets browser cookies. HttpOnly (no JS access) and Secure (HTTPS only) flags important.

🎯 INTERVIEW QUESTIONS YOU WILL BE ASKED
Q1: What is the difference between 401 and 403 HTTP status codes?
Q2: What does a 502 Bad Gateway mean? How would you debug it?
Q3: Explain how HTTPS works at a high level. What is TLS?
Q4: What is CORS? Why does it exist? How do you fix a CORS error?
Q5: What is the X-Forwarded-For header and why does it matter?
Q6: What is the difference between HTTP/1.1 and HTTP/2?
Q7: What is HSTS? Why is it important for security?

Chapter 5: Load Balancing — Distributing the Traffic

ANALOGY: Load balancing is like a hostess at a restaurant. You arrive, and instead of everyone crowding the same waiter, the hostess directs you to an available server. The restaurant can serve more customers because no single waiter is overwhelmed.

5.1 Why Load Balancing?

Load balancing distributes incoming network traffic across multiple backend servers. Goals: high availability (if one server dies, others serve traffic), scalability (add more servers easily), and performance (no single server is a bottleneck).

5.2 Load Balancing Algorithms

Algorithm	How It Works & Best For
Round Robin	Request 1→Server1, Request 2→Server2, Request 3→Server3, repeat. Simple. Best when all servers are identical.
Weighted Round Robin	Like round robin but servers get different weights. Server1 gets 70% traffic, Server2 gets 30%. Use when servers have different capacity.
Least Connections	Next request goes to server with fewest active connections. Best for long-lived connections (WebSocket, file uploads).
IP Hash / Sticky Sessions	Client's IP determines which server. Same client always hits same server. Required for stateful apps that don't use external session stores.
Random	Random server selection. Simple, roughly equivalent to round robin.
Least Response Time	Routes to server with lowest latency AND fewest connections. Most intelligent but needs monitoring.
Resource Based	Routes based on actual server CPU/memory. Requires health agents. Most advanced.

5.3 Layer 4 vs Layer 7 Load Balancing

Type	How It Works & When To Use
Layer 4 (Transport)	Routes based on IP and TCP/UDP port. Fast, simple. Cannot inspect HTTP content. Use for non-HTTP protocols or when maximum throughput is needed. AWS: Network Load Balancer (NLB).
Layer 7 (Application)	Routes based on HTTP content — URL path, headers, cookies, hostname. More intelligent. Can do SSL termination, content-based routing, A/B testing. AWS: Application Load Balancer (ALB). Most common for web apps.

Layer 7 Routing Rules Example

```
# AWS ALB Listener Rules (Path-based routing):
IF path starts with /api/    → Route to target group: api-servers
```

```
IF path starts with /static/ → Route to target group: cdn-origin
IF path starts with /ws/     → Route to target group: websocket-servers
IF header X-Version = v2     → Route to target group: v2-canary
DEFAULT                      → Route to target group: frontend
```

5.4 Health Checks

Load balancers continuously probe backend servers. If a server fails health checks, it's removed from rotation automatically. When it recovers, it's added back.

⚡ CHEAT SHEET: Health Check Configuration	
Protocol	HTTP/HTTPS most common. TCP for non-HTTP services.
Path	/health or /healthz — your app MUST expose this endpoint. Return 200 OK.
Interval	How often to check (e.g., every 30 seconds).
Threshold	How many consecutive failures before marking unhealthy (e.g., 3 failures).
Timeout	How long to wait for response (e.g., 5 seconds).

```
# Example health check endpoint (Node.js):
app.get('/health', (req, res) => {
  // Check DB connection, cache, dependencies
  if (dbConnected && cacheConnected) {
    res.status(200).json({ status: 'healthy', uptime: process.uptime() });
  } else {
    res.status(503).json({ status: 'unhealthy', db: dbConnected });
  }
});
```

5.5 SSL/TLS Termination

SSL termination means the load balancer decrypts HTTPS traffic and forwards plain HTTP to backends. This offloads CPU-intensive crypto from app servers. Communication between LB and backends is on a private network — generally OK, but add TLS there too for compliance.

AWS LOAD BALANCERS	ALB (Application) = Layer 7, HTTP/HTTPS/WebSocket, path routing, WAF integration. NLB (Network) = Layer 4, TCP/UDP/TLS, ultra-low latency, static IP. GLB (Gateway) = Layer 3, for transparent inspection appliances (firewalls, IDS/IPS).

🎯 INTERVIEW QUESTIONS YOU WILL BE ASKED
Q1: What is the difference between Layer 4 and Layer 7 load balancing?
Q2: What load balancing algorithm would you use for a stateful application?
Q3: Explain what SSL termination means and why it's used.

Q4: What happens when a backend server fails a health check?
Q5: What is the difference between AWS ALB and NLB? When do you use each?
Q6: How would you implement A/B testing or canary deployments with a load balancer?
Q7: What is a sticky session and what problems can it cause?

Chapter 6: Firewalls — Network Security Gates

ANALOGY: A firewall is a security guard at a building's entrance. They check everyone coming in (and sometimes going out) against a list of rules: 'Are you allowed here? Where are you going? What are you carrying?' If you match the rules, you pass. If not, you're denied.

6.1 Types of Firewalls

Type	How It Works
Packet Filtering	Checks IP/port headers only. Fast but no context. Rules: allow/deny by src IP, dst IP, port, protocol. Stateless — doesn't track connections.
Stateful Inspection	Tracks connection state. Knows if a packet is part of an established connection. Allows return traffic automatically. Most common today.
Application Layer (WAF)	Inspects HTTP content — SQL injection, XSS, malformed requests. Understands the actual application protocol. AWS WAF, Cloudflare WAF.
NGFW (Next-Gen)	Combines stateful + application awareness + IDS/IPS + user identity. Palo Alto, Fortinet, etc.

6.2 AWS Security Groups vs NACLs

This is one of the most common AWS interview topics. Know the difference cold.

Feature	Security Groups vs NACLs
Level	Security Groups: Instance-level (attached to ENI/EC2/RDS). NACLs: Subnet-level (all traffic in/out of subnet).
State	Security Groups: STATEFUL — allow inbound, return traffic auto-allowed. NACLs: STATELESS — must explicitly allow both inbound AND outbound.
Rules	Security Groups: Allow rules ONLY. NACLs: Allow AND Deny rules.
Evaluation	Security Groups: ALL rules evaluated, most permissive wins. NACLs: Rules evaluated in number order, first match wins.
Default	Security Groups: Default DENY all inbound, allow all outbound. NACLs: Default ALLOW all in both directions.
Use Case	Security Groups: Fine-grained per-instance control. NACLs: Subnet-wide emergency blocks, defense-in-depth.

Security Group Rule Example

```
# Web Server Security Group:
INBOUND:
  HTTP    (80)   from 0.0.0.0/0        # Allow all web traffic
  HTTPS  (443)   from 0.0.0.0/0        # Allow all HTTPS
  SSH     (22)   from 10.0.0.0/8       # Only from internal/VPN IPs

OUTBOUND:
```

```
  All traffic to 0.0.0.0/0           # Allow all outbound (default)

# App Server Security Group (receives from web server only):
INBOUND:
  Custom TCP (8080) from sg-web-server-id  # Only from web SG!

# This is called Security Group referencing — more secure than IP-based rules.
```

6.3 Linux iptables / nftables

Linux's built-in firewall. Every DevOps engineer should understand the basics even if you rarely configure it directly (Docker and Kubernetes heavily manipulate iptables automatically).

```
# View all iptables rules
iptables -L -n -v --line-numbers

# Allow SSH from specific IP
iptables -A INPUT -p tcp --dport 22 -s 192.168.1.100 -j ACCEPT

# Block all other SSH
iptables -A INPUT -p tcp --dport 22 -j DROP

# Allow established/related connections back in
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# WARNING: Docker heavily modifies iptables!
# When you run a container with -p 80:80, Docker adds NAT and FORWARD rules.
# Manual iptables rules may conflict — use Docker's --iptables=false with caution.
```

🎯 INTERVIEW QUESTIONS YOU WILL BE ASKED
Q1: What is the difference between a stateful and stateless firewall?
Q2: What is the difference between AWS Security Groups and NACLs?
Q3: If a request is blocked by a security group, what HTTP/TCP response does the client get?
Q4: What is a WAF and how does it differ from a network firewall?
Q5: How does Docker interact with iptables?
Q6: If you need to block a specific IP range from reaching ALL instances in a subnet, do you use a Security Group or NACL?
Q7: What is the principle of least privilege in the context of firewall rules?

Chapter 7: VPN & Tunneling — Private Channels

ANALOGY: A VPN is like a private underground tunnel through a public city. On the streets (internet), everyone can see you. But if you go through the tunnel, you travel privately — no one can see where you're going, and you emerge inside the private building (corporate network/cloud VPC).

7.1 What is a VPN?

A VPN (Virtual Private Network) creates an encrypted tunnel over a public network (internet) that makes remote users/offices appear to be on the same private network. It provides privacy, security, and access to private resources.

7.2 VPN Types

Type	Use Case
Remote Access VPN	Individual user connects to corporate network from home/coffee shop. OpenVPN, WireGuard, Cisco AnyConnect.
Site-to-Site VPN	Connects two entire networks (offices, data centers, AWS VPCs). Always-on. IPsec most common.
Client VPN	AWS-managed remote access VPN. Users connect to AWS VPC. TLS-based.
SSL/TLS VPN	Runs over HTTPS (port 443). Firewalls can't easily block it. Good for corporate users.

7.3 VPN Protocols

Protocol	Characteristics
OpenVPN	Open source. TLS-based. Runs on UDP/TCP port 1194 (or 443). Very flexible and secure. Widely used.
WireGuard	Modern, fast, tiny codebase. Uses state-of-the-art cryptography. Far fewer lines of code than OpenVPN = smaller attack surface.
IPsec	Suite of protocols. Two modes: Transport (encrypts payload only) and Tunnel (encrypts entire packet). Used heavily for site-to-site. Complex but very established.
IKEv2	Key exchange for IPsec. Fast reconnect — great for mobile (changes IP often).
L2TP/IPsec	Combination. L2TP provides tunneling, IPsec provides encryption. Legacy.
PPTP	Old, broken, insecure. Never use in production. MS-CHAPv2 is cracked.

7.4 Tunneling Protocols

Tunneling encapsulates one protocol inside another. VPNs use tunneling, but tunneling has other uses too.

⚡ CHEAT SHEET: Tunneling Types DevOps Engineers Use	
SSH Tunneling	Forward local port over SSH to remote host. 'ssh -L 5432:db-host:5432 bastion' → Access DB locally.
SSH SOCKS Proxy	ssh -D 1080 bastion → Creates SOCKS5 proxy. Route browser through bastion.
GRE (Generic Routing Encapsulation)	Encapsulates any Layer 3 protocol. No encryption itself — pair with IPsec.
VXLAN	Encapsulates Layer 2 in UDP. Used by Kubernetes/Docker for overlay networks between hosts.
IP-in-IP	Encapsulates IP in IP. Used in cloud provider routing.

```
# SSH Port Forwarding Examples (critical for DevOps):

# Local forwarding: Access private DB via bastion
ssh -L 5432:private-db.internal:5432 ec2-user@bastion-public-ip
# Now: psql -h localhost -p 5432   (actually connects to private DB)

# Remote forwarding: Expose local service to remote server
ssh -R 8080:localhost:3000 ec2-user@remote-server
# Remote server can now curl localhost:8080 → your local port 3000

# Dynamic forwarding: SOCKS proxy
ssh -D 1080 -N ec2-user@bastion
# Configure browser to use SOCKS5 proxy 127.0.0.1:1080
```

7.5 AWS VPN Options

Service	Use Case
AWS Site-to-Site VPN	IPsec VPN between on-premises network and AWS VPC. 2 tunnels for HA. Up to 1.25 Gbps.
AWS Client VPN	Remote users access AWS VPCs. OpenVPN-based. Managed service.
AWS Direct Connect	NOT a VPN — dedicated physical fiber from your data center to AWS. No internet. Consistent latency, higher bandwidth.
Transit Gateway + VPN	Hub-and-spoke model connecting multiple VPCs and on-premises networks.

🎯 INTERVIEW QUESTIONS YOU WILL BE ASKED
Q1: What is the difference between a site-to-site VPN and a remote access VPN?
Q2: How does SSH port forwarding work? Give a practical example.
Q3: What is the difference between IPsec Transport mode and Tunnel mode?
Q4: What is WireGuard and why is it considered more secure than OpenVPN?
Q5: What is AWS Direct Connect? How is it different from a VPN?
Q6: What is VXLAN and why is it used in container networking?

Q7: What happens when a VPN connection drops mid-session?

Chapter 8: Proxies — The Middlemen

ANALOGY: A proxy is like a personal assistant. Instead of you calling a company directly (revealing your identity and going through their queue), your assistant calls on your behalf. Or in reverse — instead of a company receiving calls from millions of customers directly, a receptionist handles all calls first, filtering and routing them appropriately.

8.1 Forward Proxy vs Reverse Proxy

Type	Direction & Use Case
Forward Proxy	Client-side proxy. Sits in FRONT of clients. Clients → Proxy → Internet. Hides client identity from servers. Used for: corporate web filtering, anonymization, caching client requests, bypassing geo-restrictions.
Reverse Proxy	Server-side proxy. Sits in FRONT of servers. Internet → Proxy → Backend Servers. Hides server identity from clients. Used for: load balancing, SSL termination, caching, WAF, API gateway functionality. NGINX, HAProxy, Envoy are reverse proxies.

KEY INSIGHT Most DevOps engineers deal with REVERSE proxies daily. When you configure NGINX to serve your app, NGINX is acting as a reverse proxy — it receives requests, applies rules, and forwards to your app server.

8.2 NGINX as a Reverse Proxy

```
# nginx.conf — Basic reverse proxy setup
upstream backend {
    least_conn;                          # Load balancing algorithm
    server app1.internal:8080;
    server app2.internal:8080;
    server app3.internal:8080;
    keepalive 32;                        # Keep 32 connections open
}

server {
    listen 443 ssl http2;
    server_name api.example.com;

    ssl_certificate     /etc/ssl/cert.pem;
    ssl_certificate_key /etc/ssl/key.pem;

    location / {
        proxy_pass http://backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 10s;
        proxy_send_timeout    30s;
        proxy_read_timeout    30s;
    }
```

```
}
```

8.3 Proxy Types

Proxy Type	Description
HTTP Proxy	Understands HTTP. Can cache, filter, modify HTTP traffic.
SOCKS Proxy	Generic proxy at TCP level. Doesn't understand the protocol — just forwards TCP connections. More flexible.
Transparent Proxy	Client doesn't know they're going through a proxy. Corporate networks use this for monitoring/filtering.
Anonymous Proxy	Hides your IP but identifies itself as a proxy in headers.
Elite/High Anonymity Proxy	Hides your IP and doesn't reveal it's a proxy at all.

8.4 Service Mesh & Sidecar Proxies

In modern microservices on Kubernetes, every pod gets a SIDECAR proxy injected automatically. This is Envoy (used by Istio, Linkerd). The sidecar handles all network traffic in/out of the pod.

⚡ CHEAT SHEET: Service Mesh Benefits	
mTLS	Mutual TLS between all services — automatic certificate management.
Traffic Management	Canary deployments, circuit breaking, retries, timeouts — without code changes.
Observability	Automatic metrics, logs, traces for every service-to-service call.
Load Balancing	Intelligent, topology-aware load balancing at the application layer.

ENVOY PROXY	Envoy is the de facto standard sidecar proxy. Istio, AWS App Mesh, Consul Connect, and others all use Envoy. Understanding Envoy = understanding modern cloud networking.

🎯 INTERVIEW QUESTIONS YOU WILL BE ASKED
Q1: What is the difference between a forward proxy and a reverse proxy?
Q2: What is NGINX and what role does it play in web infrastructure?
Q3: What is a sidecar proxy and why is it used in Kubernetes?
Q4: How do you pass the client's real IP to your application when behind a reverse proxy?
Q5: What is a service mesh and what problem does it solve?
Q6: What is the difference between SOCKS and HTTP proxies?
Q7: What is circuit breaking and how does a proxy implement it?

Chapter 9: CDN — Content Delivery Networks

ANALOGY: A CDN is like having warehouse branches in every city. Instead of everyone ordering from ONE central warehouse (origin server in Virginia), customers in Tokyo, London, and São Paulo all get their packages from local warehouses. Faster delivery, less strain on the central warehouse.

9.1 How CDN Works

A CDN is a globally distributed network of servers (Points of Presence / PoPs) that cache your content close to users. When a user requests your site, they're served from the nearest PoP rather than your origin server.

```
Without CDN:
User in Tokyo → Origin server in Virginia → 180ms latency

With CDN:
User in Tokyo → CDN PoP in Tokyo → 5ms latency  (cache hit)
              → CDN checks cache → MISS → fetches from Virginia → caches in Tokyo
Next user in Tokyo → CDN PoP in Tokyo → 5ms (cache hit!)

Cache HIT  = CDN has the content. Served instantly. Origin server NOT contacted.
Cache MISS = CDN doesn't have it. Fetches from origin, caches, then serves.
```

9.2 What CDNs Cache

Content Type	CDN Cacheable?
Static files (HTML, CSS, JS)	YES — perfect for CDN. Long TTL.
Images, videos, fonts	YES — major bandwidth savings. Long TTL.
API responses	YES — if responses are the same for all users. Short TTL.
User-specific data	NO — different for each user. Don't cache (or use Vary/Cookie headers).
Authenticated content	NO by default — vary by Authorization header.
Real-time data	NO — cache-control: no-store or short TTL.

9.3 Cache Control Headers

⚡ CHEAT SHEET: Cache-Control Directives	
max-age=3600	Cache this response for 3600 seconds (1 hour).
s-maxage=86400	CDN-specific max-age. Overrides max-age for CDN caches only.
no-cache	Must revalidate with origin before serving from cache. Can still cache.
no-store	NEVER cache this. Every request goes to origin. Use for sensitive data.

public	Can be cached by CDN (even if it looks private). Safe for shared caches.
private	Only browser can cache — NOT CDN. For user-specific data.
immutable	Content NEVER changes. Browser won't revalidate even on reload. Use with versioned filenames.
must-revalidate	Once expired, must check with origin. No stale content served.
stale-while-revalidate=60	Serve stale content for 60s while fetching fresh in background.

9.4 CDN Providers & Features

Provider / Feature	Details
Cloudflare	Largest network (270+ PoPs). Free tier. DDoS protection built-in. Workers for edge compute.
AWS CloudFront	Tightly integrated with AWS. Origins: S3, ALB, EC2, custom. Lambda@Edge for edge logic.
Fastly	Highly customizable VCL. Instant purge. Preferred by tech-savvy teams.
Akamai	Oldest, largest enterprise CDN. Global reach. Most expensive.
CDN Invalidation	Purging cached content before TTL expires. Critical after deployments. CloudFront: create invalidation. Cloudflare: purge cache.
Edge Computing	Run code at CDN edge nodes. CloudFront Lambda@Edge, Cloudflare Workers. A/B testing, auth, redirects at edge.

DEPLOYMENT TIP	Always use content-addressed/versioned filenames (app.abc123.js) with long TTLs (1 year). On deploy, filename changes → CDN fetches fresh automatically. No manual cache invalidation needed. Old users continue getting old version until their browser cache expires.

🎯 INTERVIEW QUESTIONS YOU WILL BE ASKED
Q1: What is a CDN and how does it improve performance?
Q2: What is the difference between a cache HIT and cache MISS?
Q3: When would you NOT want to cache a response on a CDN?
Q4: What is the difference between Cache-Control: no-cache and no-store?
Q5: How would you ensure users get the latest version of your JavaScript after a deployment?
Q6: What is edge computing? How does Lambda@Edge differ from a regular Lambda function?
Q7: How would you handle cache invalidation after a critical bug fix in a cached file?

Chapter 10: Docker Networking

ANALOGY: Docker networking is like an apartment building. Each container is an apartment. Docker creates virtual hallways (networks) connecting apartments. Containers in the same hallway can talk freely. Containers in different hallways need to go through the building lobby (routing) to reach each other. The building has a front door (port mapping) where the outside world enters.

10.1 Docker Network Drivers

Driver	How It Works & When To Use
bridge (default)	Creates a virtual switch (docker0). Containers get private IPs. NAT for internet access. Port mapping exposes services. Default for standalone containers. 172.17.0.0/16 by default.
host	Container shares host's network stack directly. No network isolation. No port mapping needed. Best performance. Use carefully — security risk.
none	No networking. Completely isolated. Useful for containers that don't need network access.
overlay	Spans multiple Docker hosts. Used by Docker Swarm. Creates tunnel between hosts. Containers on different hosts can communicate.
macvlan	Assigns MAC address to container. Appears as physical device on network. Use when containers need to be directly accessible on LAN.
ipvlan	Similar to macvlan but shares host's MAC. Uses IP routing instead.

10.2 Container DNS — How Containers Find Each Other

When you create a user-defined bridge network, Docker's embedded DNS server resolves container names to IPs automatically. This is why Docker Compose services find each other by name.

```
# docker-compose.yml example
version: '3.8'
services:
  web:
    image: nginx:alpine
    ports:
      - '80:80'         # host:container port mapping
    networks:
      - frontend

  api:
    image: myapp:latest
    environment:
      - DB_HOST=postgres  # Container name resolution!
    networks:
      - frontend
      - backend

  postgres:
```

```
    image: postgres:15
    networks:
      - backend          # NOT exposed to frontend network!

networks:
  frontend:
  backend:

# 'api' can reach 'postgres' via hostname 'postgres'
# 'web' CANNOT reach 'postgres' — different network
# This is network segmentation in docker-compose!
```

10.3 Port Mapping

```
# Port mapping: -p host_port:container_port
docker run -p 80:8080 nginx       # Host port 80 → Container port 8080
docker run -p 127.0.0.1:80:8080 nginx  # Bind to localhost only (more secure)
docker run -p 8080-8090:8080-8090 nginx # Port range

# What actually happens:
# Docker adds an iptables DNAT rule:
# Any traffic to host:80 → forward to container_ip:8080

# EXPOSE in Dockerfile:
EXPOSE 8080  # Documentation only! Does NOT publish the port.
             # You still need -p flag to access from outside.
```

10.4 Docker Networking Internals

Understanding this helps you debug complex networking issues in containers:

```
# Inspect container networks
docker network ls
docker network inspect bridge
docker inspect container_name | grep -A 20 'NetworkSettings'

# Check container IP
docker inspect container_name | grep IPAddress

# Get a shell in a running container for debugging
docker exec -it container_name sh

# Inside container: check network
ip addr show
ip route show
cat /etc/resolv.conf
nslookup other-service

# Check if two containers can reach each other
docker exec container1 ping container2
docker exec container1 curl http://api-service:8080/health
```

10.5 Common Docker Networking Problems & Solutions

Problem	Cause & Solution
Container can't reach internet	Check if bridge network has internet access. Try: docker run --rm alpine ping 8.8.8.8. Often iptables issue or DNS.
Containers can't find each other by name	Using default bridge network! Switch to user-defined bridge. docker network create mynet && docker run --network mynet
Port already in use	Host port 80 already taken. Use different host port: -p 8080:80. Check: ss -tulpn | grep :80
172.17.0.0 conflicts with VPC	Docker default subnet conflicts with AWS VPC. Change in /etc/docker/daemon.json: {"bip": "192.168.100.1/24"}
Container accessible from outside unexpectedly	Binding 0.0.0.0 exposes to all interfaces. Bind to 127.0.0.1: -p 127.0.0.1:8080:80

🎯 INTERVIEW QUESTIONS YOU WILL BE ASKED
Q1: What is the difference between Docker bridge and host networking?
Q2: How do containers in the same Docker Compose file communicate with each other?
Q3: What does EXPOSE in a Dockerfile actually do? How is it different from -p?
Q4: How would you connect two separate Docker Compose projects' containers?
Q5: Why might Docker's default 172.17.0.0/16 subnet cause issues in a cloud environment?
Q6: What happens at the iptables level when you do docker run -p 80:8080?
Q7: How would you debug a situation where two containers can't communicate?

Chapter 11: Kubernetes Networking — The Complex Beast

ANALOGY: Kubernetes networking is like a city's entire infrastructure: roads (CNI), traffic lights (kube-proxy), postal addresses (DNS), city entrances (Ingress), and internal phone system (Services). Each pod is a building with its own address. Services are like post office boxes — a stable address that forwards mail to the right building even as buildings come and go.

11.1 Kubernetes Networking Model (Fundamental Rules)

Kubernetes mandates a flat networking model with 4 fundamental rules. Every CNI plugin MUST implement these:

Rule	What It Means
1. Pod-to-Pod across nodes	All pods can communicate with all other pods WITHOUT NAT, regardless of which node they're on.
2. Node-to-Pod	Nodes can communicate with all pods without NAT.
3. Pod's own IP	A pod sees the same IP that others use to reach it — no IP masquerading from the pod's perspective.
4. No port conflicts	Each pod gets its own unique IP, so there are no port conflicts between pods.

11.2 Kubernetes Service Types

Service Type	Behavior & Use Case
ClusterIP (default)	Internal-only virtual IP. Only accessible within the cluster. Used for service-to-service communication.
NodePort	Exposes service on a static port (30000-32767) on EVERY node's IP. External access possible but not recommended for production.
LoadBalancer	Creates a cloud provider load balancer (AWS ALB/NLB, GCP LB). The recommended way to expose services externally. Each service gets its own LB.
ExternalName	Maps service to a DNS name. Returns CNAME record. Used to integrate external services into cluster DNS.
Headless (ClusterIP: None)	No virtual IP. DNS returns individual pod IPs directly. Used for StatefulSets and direct pod addressing (databases).

```
# Service YAML examples
# ClusterIP - internal only
apiVersion: v1
kind: Service
metadata:
  name: my-api
spec:
  selector:
    app: my-api       # Routes to pods with this label
  ports:
    - port: 80        # Service port (what others call)
```

```
      targetPort: 8080 # Pod port (where app listens)
  type: ClusterIP

# DNS: my-api.default.svc.cluster.local
# Or just: my-api  (within same namespace)
```

11.3 Ingress — The Gateway Controller

Instead of creating one LoadBalancer per service (expensive!), Ingress exposes multiple services through a SINGLE entry point using routing rules.

```
# Ingress routes by hostname and path
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: main-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  tls:
    - hosts:
        - api.example.com
      secretName: tls-secret      # TLS cert stored as Secret
  rules:
    - host: api.example.com
      http:
        paths:
          - path: /v1
            pathType: Prefix
            backend:
              service:
                name: api-v1
                port: number: 80
          - path: /v2
            pathType: Prefix
            backend:
              service:
                name: api-v2
                port: number: 80
```

11.4 CNI Plugins

CNI (Container Network Interface) plugins implement the Kubernetes networking model. They create and manage the network interfaces that give pods their IPs and enable cross-node communication.

CNI Plugin	Characteristics
Flannel	Simplest. VXLAN or host-gw. No NetworkPolicy support. Good for learning.
Calico	Most popular enterprise CNI. BGP-based routing. Full NetworkPolicy. eBPF dataplane option. Used in many production clusters.
Weave	Mesh network. Automatic encryption. Simpler setup.

Cilium	eBPF-based. Outstanding observability. Layer 7 NetworkPolicy (HTTP-aware). High performance. Growing fast.
AWS VPC CNI	Pods get actual VPC IPs. Native AWS routing — no overlay. Direct integration with Security Groups.

11.5 Network Policies

By default, ALL pods can communicate with ALL other pods. NetworkPolicy adds firewall rules for pods. Only works if your CNI supports it.

```
# Deny all ingress to a namespace, then selectively allow
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all-ingress
  namespace: production
spec:
  podSelector: {}        # Applies to ALL pods in namespace
  policyTypes:
    - Ingress
  # No ingress rules = deny all ingress

---
# Allow only frontend to call api
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-api
spec:
  podSelector:
    matchLabels:
      app: api
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend
      ports:
        - protocol: TCP
          port: 8080
```

11.6 kube-proxy

kube-proxy runs on every node and implements the Service abstraction. When you create a Service with ClusterIP, kube-proxy creates iptables rules (or IPVS rules) to forward traffic from the ClusterIP to actual pod IPs.

HOW SERVICE IPs WORK	ClusterIP is a VIRTUAL IP — no actual network interface has this IP. kube-proxy creates iptables DNAT rules: 'Any traffic to 10.96.100.5:80 → randomly choose one of {pod1:8080, pod2:8080, pod3:8080}'. That's your Service load balancing!

11.7 DNS in Kubernetes (CoreDNS)

```
# Pod DNS resolution order:
1. /etc/hosts  (in the pod)
2. CoreDNS (in-cluster DNS server at kube-dns service IP, usually 10.96.0.10)
3. External DNS (forwarded by CoreDNS)

# Full DNS name format:
<service>.<namespace>.svc.<cluster-domain>
# cluster-domain is typically: cluster.local

# Examples:
mysql.database.svc.cluster.local    # From any namespace
mysql.database                       # Short form from any namespace (search
domain)
mysql                                # Only from 'database' namespace itself

# Pod DNS:
10-0-1-5.default.pod.cluster.local  # Pod with IP 10.0.1.5 in default namespace

# Check DNS inside pod:
kubectl exec -it mypod -- nslookup kubernetes.default
kubectl exec -it mypod -- cat /etc/resolv.conf
```

🎯 INTERVIEW QUESTIONS YOU WILL BE ASKED
Q1: What are the 4 fundamental rules of Kubernetes networking?
Q2: What is the difference between ClusterIP, NodePort, and LoadBalancer services?
Q3: What is an Ingress? What is an Ingress Controller?
Q4: What is a CNI plugin? Name three CNI plugins.
Q5: How does kube-proxy implement Kubernetes Services?
Q6: What is a NetworkPolicy and what is its default behavior (without any policy)?
Q7: What DNS name would you use to reach service 'api' in namespace 'backend'?
Q8: What is a headless service and when would you use it?
Q9: How does the AWS VPC CNI differ from overlay-based CNIs like Flannel?

Master Quick Reference — The Essentials

⚡ CHEAT SHEET: The Numbers Every DevOps Engineer Must Know	
IPv4 = 32 bits	4 octets, ~4.3 billion addresses
IPv6 = 128 bits	8 groups of 4 hex, ~340 undecillion addresses
/24 = 256 IPs	254 usable hosts. Your most common subnet.
/16 = 65,536 IPs	Docker default bridge, AWS VPC typical.
Port 22 = SSH	Always remember this. Encrypted remote access.
Port 80/443	HTTP and HTTPS. Insecure/Secure web.
Port 53 = DNS	UDP for queries, TCP for zone transfers.
TCP 3-way handshake	SYN → SYN-ACK → ACK
HTTP 200/201/204	OK / Created / No Content
HTTP 301/302	Permanent redirect / Temporary redirect
HTTP 401/403	Unauthorized (no creds) / Forbidden (wrong perms)
HTTP 502/503/504	Bad Gateway / Unavailable / Gateway Timeout
NodePort range	30000–32767 in Kubernetes
Kubernetes DNS	<svc>.<ns>.svc.cluster.local
Docker default subnet	172.17.0.0/16
Private IP ranges	10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16

YOU ARE NOW A NETWORKING PRO.

This knowledge lives in you. Study it. Apply it. You are ready.